New Relic APM — Day 3 Study Guide: Performance Optimization and Dashboards

Oct 29, 2026 · @Vivek Arora

Day 3 at a glance

Days 1 and 2 were about *finding* problems and *responding* to them. Day 3 is about the causes that hide underneath slow transactions — the **JVM**, the **host** and the **database** — and about the one screen an on-call engineer opens first: a **dashboard** built for production support.

Agenda

Time	Session	Lab
09:30 – 09:45	Day 2 recap; extend the lab system	—
09:45 – 10:45	**Module 7** — JVM, runtime and infrastructure metrics	
10:45 – 11:00	Break	
11:00 – 12:00	Lab 3.1 — Diagnose a GC-driven response-time degradation	Lab 3.1 (60 min)
12:00 – 13:00	**Module 8** — Database monitoring and query optimization	
13:00 – 14:00	Lunch	
14:00 – 15:15	Lab 3.2 — Identify and explain the top slow SQL statement	Lab 3.2 (75 min)
15:15 – 15:30	Break	
15:30 – 16:15	**Module 9** — Production support dashboards	
16:15 – 17:15	Lab 3.3 — Build an on-call command-center dashboard	Lab 3.3 (60 min)
17:15 – 17:30	Course wrap-up, knowledge check, Q&A	—

Learning objectives

By the end of Day 3 you can:

1. Read thread counts, thread-pool and connection-pool usage, and recognise **saturation** — requests waiting for a thread or a connection.
2. Read heap usage and **garbage-collection** time, tell a healthy sawtooth from a **memory leak**, and prove that GC is driving response time.
3. Correlate host and container CPU and memory with APM response time, and spot capacity warning signs in trends.
4. Use the APM **Databases** page, **slow query traces** and **explain plans** to find and explain the statement that costs a critical transaction the most time.
5. Choose the right fix — an index, removing an N+1, batching, or pool tuning — and verify it with before/after data.
6. Design an on-call dashboard around the **golden signals**, with billboards, thresholds, template variables, linked facets and drill-downs, and keep it healthy with sharing and naming rules.

How to use this guide with the lab guide

This study guide explains the concepts; every command, click and expected screen is in the separate New Relic APM — Day 3 Lab Guide. Each module ends with a *"Do Lab 3.x"* pointer. Screens in the lab guide are redrawn illustrations labelled **Expected view**; menu names may differ slightly in your account.

What the lab system gains today

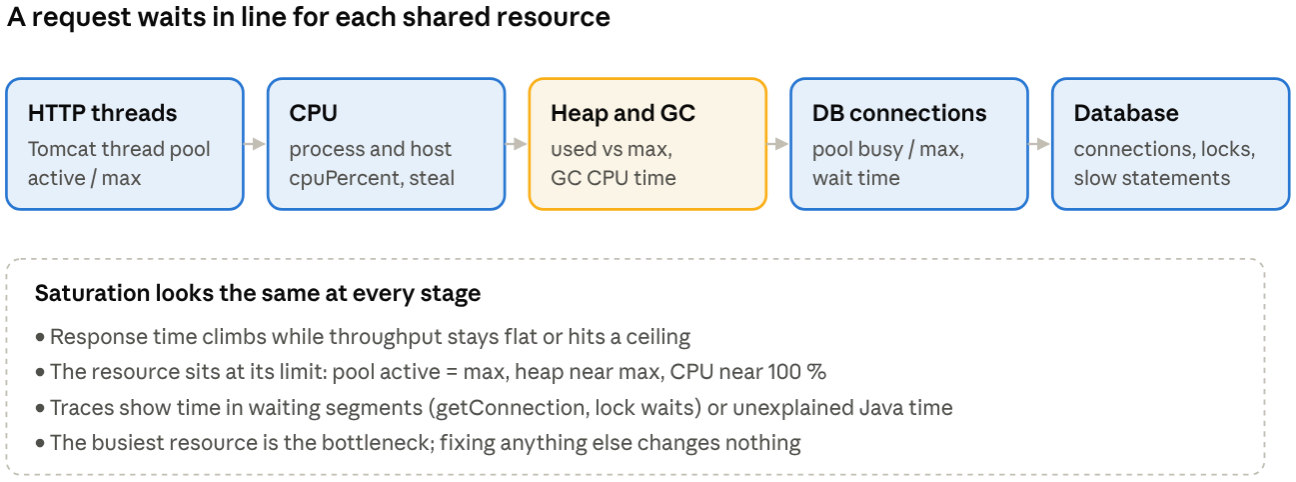
Addition	Why	Lab
New Relic infrastructure agent (container)	Host, process and container metrics next to APM: `SystemSample`, `ProcessSample`, `ContainerSample`	3.1
easyTravel pattern `LargeMemoryLeak`	A real leak in the business backend, to watch heap and GC take over response time	3.1
Spring PetClinic + PostgreSQL (Java agent, `petclinic`)	A SQL database, so the Java agent can capture slow SQL and **PostgreSQL explain plans**; 2 million owners make one query slow	3.2

Addition	Why	Lab
PostgreSQL on-host integration	Database instance views: connections, commits, rows, buffer hits, query-level metrics	3.2

Together, `easyTravel-frontend`, `easyTravel-backend` and `petclinic` are the three services of the "microservice application" the Lab 3.3 dashboard watches.

Module 7 — JVM, runtime and infrastructure metrics

A transaction trace tells you *where* a request spent its time. When the time is "in Java" but no single method explains it, or when everything gets slower at once, the cause is usually a **shared resource** the request had to wait for: a thread, the CPU, memory, or a database connection. Module 7 is about reading those resources.



The shared resources behind every transaction, and the signs that one is saturated

7.1 Threads, thread pools and connection-pool saturation

A Java web service handles each request on a **thread** from a pool (Tomcat's HTTP connector pool, 200 threads by default). Most requests also borrow a **database connection** from a connection pool (Tomcat JDBC, HikariCP — 10 connections by default in Spring Boot). Both pools are queues: when every thread or connection is busy, new requests wait.

Little's law connects the numbers: *busy workers = throughput × time each request holds one*. At 50 requests/s and 0.2 s per request you need 10 busy threads; if the database slows to 1 s, you need 50 — and a pool of 20 connections saturates.

Where to look:

Signal	Where in New Relic	Saturated when…
Thread count	APM → service → **JVMs** → *Threads*	Keeps climbing (thread leak), or flat at the pool maximum
Thread-pool active / idle / *active-to-max*	**JVMs** page (thread pools, from JMX)	Active = max for minutes; idle = 0
Connection-pool active / idle / max	**JVMs** page (*Data sources*, for Tomcat pools) or the pool's metrics (`Database Connection/…` for HikariCP)	Active = max; requests show *getConnection* time in traces
Response time vs throughput	APM **Summary**	Response time up, throughput flat or capped
Where threads are stuck	APM → **Thread profiler** (Java)	Many samples in the same wait / lock frame

The classic saturation fingerprint: **throughput stops growing while response time climbs**, CPU is moderate, and the extra time is *waiting* — not doing.

For Tomcat with Spring Boot 2.2+, JMX MBeans (and so the JVMs page's thread-pool charts) need `server.tomcat.mbeanregistry.enabled=true`.

7.2 Garbage collection, heap usage and memory-leak patterns

Java objects live on the **heap**. The garbage collector (G1 by default on modern JVMs) frees unreachable objects: frequent, short **young** collections for short-lived objects, and occasional **old-generation** or **full** collections for long-lived ones. Collections cost CPU, and some pause application threads.

Pattern	Heap chart	GC time	What it means
Healthy	Sawtooth: rises, drops after each GC, the *floor* stays level	Low (a few % of CPU)	Normal allocation
Undersized heap	Sawtooth close to the maximum, floor level	High and steady	Too little headroom for the live data — raise `-Xmx` (and the container limit)
Memory leak	The floor after each GC **rises** over time until it meets the maximum	Rising, then dominant	Objects are kept alive by mistake; ends in `OutOfMemoryError` or a container kill

Pattern	Heap chart	GC time	What it means
Allocation storm	Steep teeth, very frequent GCs	Spiky	Code creating huge numbers of short-lived objects (a hot loop, large result sets)

How GC drives response time: when the heap is nearly full, the JVM spends a growing share of CPU in GC and pauses threads more often. Every request gets slower, the slowdown is spread across **all** transactions (not one), and the transaction traces show time "in Java" with no slow segment to blame. The JVMs page shows the cause: **heap used approaching the max** and **GC CPU time rising** at the same moment response time rises.

```
-- 1. Find the JVM metric names your agent reports
SELECT uniques(metricTimesliceName) FROM Metric
WHERE appName = 'easyTravel-backend'
  AND (metricTimesliceName LIKE 'Memory/%' OR metricTimesliceName LIKE 'GC/%'
       OR metricTimesliceName LIKE 'Threads/%' OR metricTimesliceName LIKE
'JmxBuiltIn/%')
SINCE 30 minutes ago

-- 2. Heap used (MB) over time
SELECT average(newrelic.timeslice.value) AS 'heap used (MB)' FROM Metric
WHERE appName = 'easyTravel-backend' AND metricTimesliceName =
'Memory/Heap/Used'
TIMESERIES 1 minute SINCE 1 hour ago

-- 3. Time spent in each collector
SELECT sum(newrelic.timeslice.value) FROM Metric
WHERE appName = 'easyTravel-backend' AND metricTimesliceName LIKE 'GC/%'
FACET metricTimesliceName TIMESERIES 1 minute SINCE 1 hour ago
```

New Relic shows the **symptom and timing**; finding *which* objects leak needs a heap dump (`jcmd <pid> GC.heap_dump`) and a heap analyser. The fix is in code; the short-term mitigation is a restart and more headroom.

In containers, size the heap to the container: a container limit of 1 GB with `-Xmx1g` leaves no room for metaspace, threads and buffers, and the kernel kills the process. Prefer `-XX:MaxRAMPercentage=70` or an explicit `-Xmx` well under the limit.

7.3 CPU, memory and process metrics: host health next to APM

The **infrastructure agent** adds the host's view:

Event type	Key attributes	Question it answers
`SystemSample`	`cpuPercent`, `cpuStealPercent`, `memoryUsedPercent`, `loadAverageOneMinute`	Is the host itself busy or short of memory?
`ProcessSample`	`processDisplayName`, `cpuPercent`, `memoryResidentSizeBytes`, `threadCount`	Which process uses it — our JVM or a neighbour?
`ContainerSample`	`name`, `cpuPercent`, `memoryUsageBytes`, `memorySizeLimitBytes`, `restartCount`	Is a container near its limits, or being restarted?
`StorageSample` / `NetworkSample`	`diskUsedPercent`; `receiveBytesPerSecond`	Disk full? Network saturated?

APM services are linked to the hosts and containers they run on, so the service page shows the host's health beside response time. Correlate in NRQL:

```
-- Backend response time next to its container's CPU and memory
SELECT average(duration) * 1000 AS 'avg ms' FROM Transaction
WHERE appName = 'easyTravel-backend' TIMESERIES 1 minute SINCE 1 hour ago

SELECT average(cpuPercent) AS 'cpu %', average(memoryUsageBytes) / 1e6 AS
'memory MB'
FROM ContainerSample WHERE name = 'backend' TIMESERIES 1 minute SINCE 1 hour
ago

-- Top processes on the host
SELECT average(cpuPercent), average(memoryResidentSizeBytes) / 1e6 AS 'RSS
MB'
FROM ProcessSample FACET processDisplayName SINCE 30 minutes ago LIMIT 10
```

Read them together: **host CPU high + JVM GC time high** → the JVM is burning CPU in GC; **host CPU high + JVM GC low + another process high** → a noisy neighbour; **CPU steal high** → the virtual machine is starved by its hypervisor; **container memory near the limit + restarts** → the heap or native memory is too large for the container.

7.4 Application health over time: trends and capacity warning signs

Incidents are often announced weeks ahead in slowly moving lines. Look at trends across days and weeks, not minutes:

Warning sign	Query idea
Response time creeping up at the same throughput	`SELECT percentile(duration, 95) … SINCE 4 weeks ago TIMESERIES 1 day`
Heap floor after GC rising release by release	Heap used per day, faceted by `tags.service.version` where available
Peak throughput approaching a known ceiling	`SELECT max(rpm) FROM (SELECT rate(count(*), 1 minute) AS rpm … TIMESERIES 1 minute) SINCE 1 week ago`
Pools running hotter every week	Connection-pool *active/max* per day
Disk, memory or CPU trending to a limit	`SELECT predictLinear(diskUsedPercent, 1 week) FROM StorageSample FACET hostname`

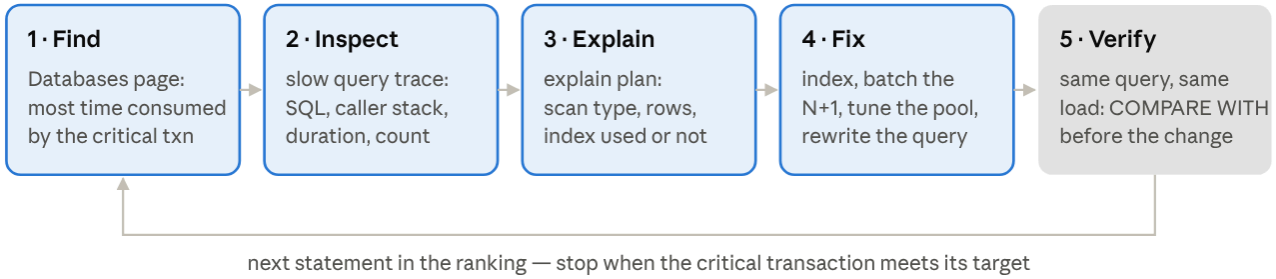
Use `COMPARE WITH 1 week ago` for the same hour last week, and metrics (`FROM Metric`) rather than events for ranges longer than your event retention.

> **Do Lab 3.1 — Diagnose a GC-driven response-time degradation** in the Day 3 Lab Guide (60 min): install the infrastructure agent, record the backend's healthy JVM baseline, switch on `LargeMemoryLeak` , and use the JVMs page, metric timeslices and container metrics to prove that garbage collection — not code or the database — is driving response time.

Module 8 — Database monitoring and query optimization

Most slow transactions are slow in the database: one expensive statement, the same cheap statement repeated hundreds of times, or requests waiting for a connection. Module 8 shows how New Relic separates those cases and how to prove a fix.

The query optimization loop: rank by total time, then explain, fix, verify



Total time = average time × calls. A 5 ms query run 200 times per request outranks a 300 ms query run once a day.

The loop used in Lab 3.2

8.1 Database transactions and the database breakdown

The Java agent instruments JDBC and many datastore clients, and records each call as a **datastore segment** named by database product, table (or collection) and operation — for example `Datastore/statement/Postgres/owners/select`.

View	Shows	Use it to
APM **Summary** stacked chart	Database layer vs application code over time	See *whether* the database is the problem
Transaction **breakdown table**	% time, calls and time per call for each table/operation in one transaction	See *which* statement matters for *that* transaction
APM **Databases** page	Every operation ranked by **most time consumed**, slowest average, or throughput; the transactions that call it	Rank statements across the service
Transaction **traces**	Each query in order, with duration and call count	Spot N+1 and repeated calls
Spans (`category = 'datastore'`)	`db.system` , `db.statement` (obfuscated), table, `peer.hostname` , duration	Query across services and traces with NRQL

```
-- Share of each transaction's time spent in the database
SELECT average(databaseDuration) / average(duration) * 100 AS 'db %',
       average(databaseCallCount) AS 'db calls', average(duration) AS 'avg s'
FROM Transaction WHERE appName = 'petclinic' FACET name SINCE 1 hour ago

-- Slowest datastore operations across services
SELECT average(duration.ms), percentile(duration.ms, 95), count(*) FROM Span
WHERE category = 'datastore' FACET appName, name SINCE 1 hour ago LIMIT 20
```

Rank by **total time** (*most time consuming*), not by average: total = average × calls, and total is what your users wait for.

8.2 Slow query traces, explain plans and obfuscation

When the agent captures a transaction trace, it also keeps the **slowest SQL statements** from it as *slow query* samples, shown on the **Databases** page (select an operation, then a slow query). Each sample shows the statement (obfuscated by default), its duration and call count, the transaction that ran it, the **stack trace** of the code that called it, and — for **MySQL and PostgreSQL** with the Java agent — the **explain plan**.

Java agent settings (`newrelic.yml`, `transaction_tracer`):

```
transaction_tracer:
  enabled: true
  transaction_threshold: apdex_f   # traces for transactions slower than 4 x
Apdex T
  record_sql: obfuscated           # off | obfuscated | raw
  explain_enabled: true            # explain plans (MySQL and PostgreSQL)
  explain_threshold: 0.5           # seconds: statements slower than this get
a plan and a slow-query sample
  stack_trace_threshold: 0.5       # seconds: statements slower than this get
a caller stack trace
  top_n: 20
```

Obfuscation replaces literals with `?`: `SELECT … FROM owners WHERE last_name LIKE ?` instead of the customer's name. Keep `obfuscated` in production; `raw` sends personal data and secrets in literals to New Relic and is blocked by high-security mode; `off` removes SQL entirely. Obfuscation does not affect the explain plan, which the agent runs against the database itself.

Reading a PostgreSQL plan — from the innermost node outwards:

You see	It means	Usually fix with
`Seq Scan on owners` + `Filter: (last_name ~~ …)`, rows in the millions	Every row read to find a few	An index the planner **can** use for that predicate
`Index Scan` / `Index Only Scan` / `Bitmap Index Scan`	Index used	— (check row estimates)
`Sort` with `external merge Disk`	Sort spilled to disk	Index matching the `ORDER BY`, more `work_mem`
`Nested Loop` with a large outer row count	One inner lookup per outer row	Index on the join column, or a hash join
Estimated rows far from actual (with `EXPLAIN ANALYZE`)	Stale statistics	`ANALYZE` the table

A trap the lab uses: PostgreSQL cannot use an ordinary B-tree index for `LIKE 'abc%'` when the database collation is not `C` (most installations use `en_US.utf8`). The index exists, the plan still says *Seq Scan*. The fix is an index with the pattern operator class: `CREATE INDEX … ON owners (last_name text_pattern_ops)`.

8.3 Database instance views: connections, locks and throughput

APM sees the database from the application's side. The **on-host integrations** (PostgreSQL, MySQL, SQL Server, Oracle, MongoDB and others, run by the infrastructure agent) see it from the server's side and create a database **entity** with its own pages.

Question	PostgreSQL integration data
Are we near the connection limit?	`PostgresqlDatabaseSample`: `db.connections` vs `PostgresqlInstanceSample` /settings `db.maxconnections`
How busy is it?	`db.commitsPerSecond`, `db.rollbacksPerSecond`, `db.rowsFetchedPerSecond`, `db.rowsReturnedPerSecond`
Is it reading from memory or disk?	`db.bufferHitsPerSecond` vs block reads
Locks and conflicts?	Lock metrics (`COLLECT_DB_LOCK_METRICS: true`), `db.conflicts.locksPerSecond` , query monitoring `PostgresBlockingSessions`
Which statements cost the server most?	Query monitoring (`ENABLE_QUERY_MONITORING` , needs `pg_stat_statements`): `PostgresSlowQueries` , `PostgresWaitEvents` , `PostgresExecutionPlanMetrics`

The two views answer different questions: APM says *which transaction and which line of code* runs the statement; the instance view says *what it costs the server* and whether *other* clients are causing the load.

```
SELECT latest(db.connections), latest(db.commitsPerSecond),
latest(db.rowsFetchedPerSecond)
FROM PostgresqlDatabaseSample FACET database SINCE 15 minutes ago

SELECT * FROM PostgresSlowQueries SINCE 30 minutes ago LIMIT 20
```

8.4 Common fixes: indexing, N+1 queries and connection-pool tuning

Problem	Evidence in New Relic	Fix
Missing or unusable index	One statement dominates time; explain plan shows a sequential scan over many rows	Add the right index (columns in `WHERE` / `JOIN` / `ORDER BY` ; operator class for `LIKE`); verify the plan changes

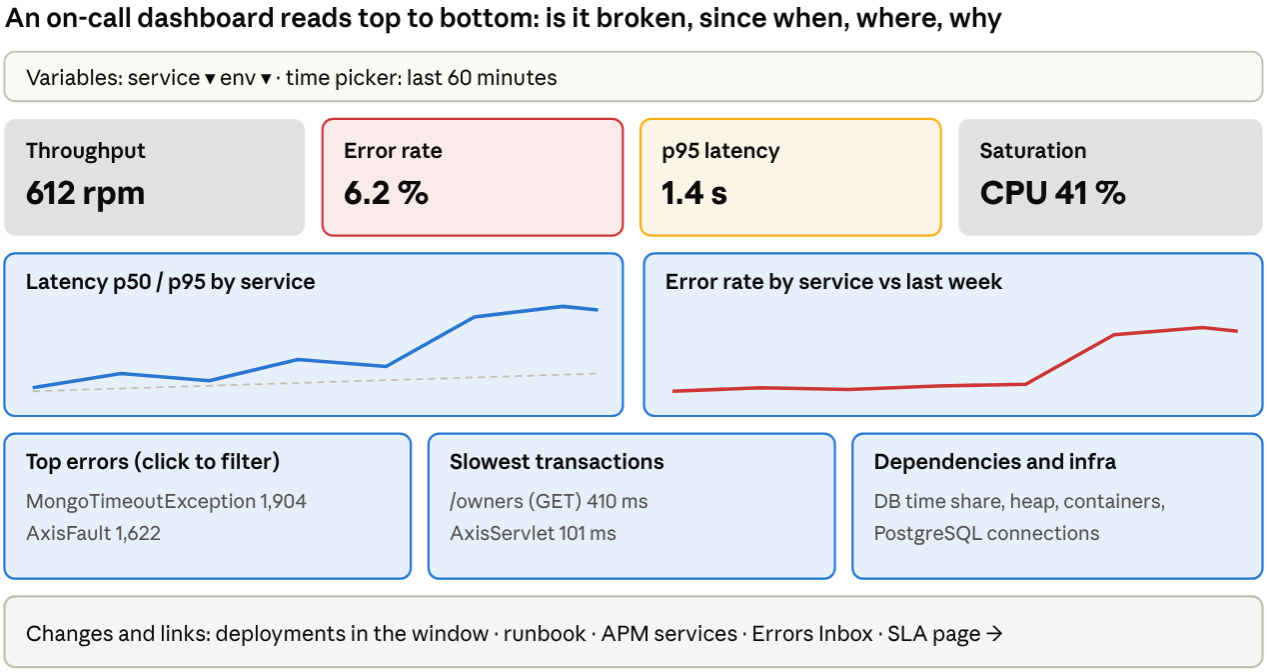
Problem	Evidence in New Relic	Fix
N+1 queries	High `databaseCallCount`; *Identical Calls* in traces; fast calls, slow total	Fetch in one query (`JOIN FETCH`, `IN (…)`, batch size), or cache
Count queries on large tables	A `select count(…)` next to every paged query, as slow as the query itself	Avoid exact counts (keyset pagination, estimated counts) or index the filter
Pool too small	Pool active = max; time in `getConnection`; database itself idle	Size the pool by Little's law; check the database's `max_connections` across all instances
Pool too large	Database CPU high, many idle connections, lock waits	Fewer connections often go *faster*; a common start is cores × 2 per database
Connection leak	Active connections climb and never fall; eventual timeouts	Close connections (`try-with-resources`); enable the pool's leak detection
Lock contention	Statements slow only under concurrency; blocking sessions	Shorter transactions, consistent lock order, better indexes for `UPDATE … WHERE`

Always verify with the same NRQL before and after (`COMPARE WITH`), at similar traffic.

> **Do Lab 3.2 — Identify and explain the top slow SQL statement** in the Day 3 Lab Guide (75 min): run Spring PetClinic on PostgreSQL with the Java agent, load 2 million owners, find the statement that costs the owner-search transaction the most time, read its slow query trace and explain plan, fix it with the right index, prove the improvement, and inspect the database instance with the PostgreSQL integration.

Module 9 — Production support dashboards

A support dashboard is not a report. It is the first screen opened when a page arrives at 3 a.m., by someone who may not know the service. It must answer, in under a minute: **is something broken, since when, where, and what changed?**



The layout built in Lab 3.3

9.1 Designing dashboards for on-call

Golden signals per service form the top row:

Signal	Typical NRQL	Threshold colour from
Traffic	`SELECT rate(count(*), 1 minute) FROM Transaction WHERE appName IN ({{service}})`	Anomaly expectations (a drop is often worse than a rise)
Errors	`SELECT percentage(count(*), WHERE error IS true) FROM Transaction …`	The same values as the error-rate alert
Latency	`SELECT percentile(duration, 95) FROM Transaction …`	The SLA / latency alert
Saturation	CPU, heap, connection-pool or container memory	Capacity limits

Rules that make the difference:

1. **Top to bottom = triage order:** golden signals → trends with comparison → breakdowns (which transaction, which error, which host) → dependencies and infrastructure → changes and links.
2. **Thresholds match the alerts.** A billboard that turns red at a different number than the pager confuses everyone.

3. **Every chart has a question in its title** ("Error rate by service vs last week"), units, and the same time window as the page.
4. **Few widgets, all actionable.** If nobody would change a decision based on it, remove it.
5. **Separate audiences.** On-call page (minutes), service owner page (days), SLA page (30 days) — pages of one dashboard, not one crowded page.

An **SLA view** answers "are we keeping our promise?" over the SLA period:

```
-- 30-day availability against a 99.5 % target
SELECT percentage(count(*), WHERE error IS false) AS 'availability %' FROM
Transaction
WHERE appName = 'easyTravel-frontend' SINCE 30 days ago

-- Requests within the latency target (1 s)
SELECT percentage(count(*), WHERE duration <= 1) AS 'within 1 s %' FROM
Transaction
WHERE appName = 'easyTravel-frontend' SINCE 30 days ago

-- Error budget left: allowed failures (0.5 %) minus actual failures
SELECT 0.5 - percentage(count(*), WHERE error IS true) AS 'error budget left
(pp)'
FROM Transaction WHERE appName = 'easyTravel-frontend' SINCE 30 days ago
```

For formal SLOs, New Relic **service levels** define SLIs from the same queries and track the budget for you; a dashboard can show them next to the raw signals.

9.2 Charts, tables, billboards and thresholds

Widget	Best for	Avoid for
Billboard	One number now, with warning/critical colours	Trends
Line	Trends, comparisons (`COMPARE WITH`), several services	More than ~6 lines
Area / stacked bar	Parts of a whole over time (time per layer, errors by class)	Comparing separate signals
Bar	Ranking facets (slowest transactions, top errors)	Time
Table	Exact values, several columns, links to drill into	Shapes

Widget	Best for	Avoid for
Histogram / heatmap	Distributions (latency spread)	Single values
Pie	Rarely: two or three shares	Anything else
Markdown	Runbook links, owners, how to read the page	Data

Thresholds: billboards take a *warning* and a *critical* value and turn amber/red when crossed (choose whether high or low is bad). Line and area charts can draw threshold lines or bands. Keep the numbers identical to the alert conditions from Day 2.

9.3 Template variables and filters for multi-service views

Variables turn one dashboard into a view of any service or environment. In edit mode, **+ Add variable**:

Type	Values from	Example
Query (NRQL)	A query returning a list	`SELECT uniques(appName) FROM Transaction SINCE 1 day ago`
List	Fixed values	`production, staging`
Text	What the viewer types	A trace ID or customer ID

Use a variable with double braces. For **multi-value** variables use `IN`, not `=`:

```
SELECT percentile(duration, 95) FROM Transaction
WHERE appName IN ({{service}}) FACET appName TIMESERIES
```

The variable's **output format** decides how the value is inserted: *String* (quoted, for `WHERE`), *Number* (for `percentile(duration, {{p}})`), *Identifier* (unquoted, for `FACET {{groupBy}}` or `FROM {{eventType}}`). Variables work in `SELECT`, `FROM`, `WHERE`, `FACET` and `ORDER BY` — not in `SINCE`, `LIMIT`, `TIMESERIES` or `COMPARE WITH`. Query-type variables use their own time range (1 hour unless the query has `SINCE`).

9.4 NRQL-driven widgets, linked facets and drill-down navigation

- **Any NRQL** is a widget: build it in the query builder, then *Add to dashboard*.

- **Linked facets**: on a faceted widget (bar, pie, table), turn on *filter the current dashboard*. Clicking a facet value — an error class, a service — filters every widget on the page to it, so the page becomes a drill-down without new queries. Facets can also link to **another dashboard**, carrying the value as its filter.
- **Entity links**: tables of services or hosts link to the entity's own pages (APM summary, traces, Errors Inbox).
- **Markdown drill-downs**: links to the runbook, the SLA page, Errors Inbox for the workload, and the alert policy; markdown can use the variables in URLs.
- **Multiple pages** keep audiences apart: *On-call*, *Service detail*, *SLA*.

9.5 Sharing, permissions and dashboard hygiene

- **Permissions** (dashboard settings): everyone in the account can edit; everyone can view but only the owner edits; or private. Support dashboards that people rely on should be *view for all, edit for owners* so they cannot drift by accident.
- **Sharing**: link to the dashboard (with its time window and variable values in the URL), export a page as PDF or snapshot for a post-incident review, or embed a single chart. Public live links make data visible outside the account — use them only for data you would publish.
- **Hygiene**: one naming convention (`[Team] Service — purpose`), a description with the owner and the last review date, tags, no personal copies of shared dashboards, and a quarterly review that deletes what nobody opened.
- **Dashboards as code**: copy a dashboard's JSON (or create it through NerdGraph `dashboardCreate`, or Terraform's New Relic provider) and keep it in Git, so a broken edit can be rolled back and the same dashboard can be stamped out per service.

> **Do Lab 3.3 — Build an on-call command-center dashboard** in the Day 3 Lab Guide (60 min): build a two-page dashboard for easyTravel and PetClinic — golden-signal billboards with thresholds, a `service` variable, trends with comparisons, linked-facet error and transaction breakdowns, JVM, database and container health, an SLA page and a runbook panel — then set its permissions and save its JSON.

Performance triage runbook and Day 3 cheat sheet

"Everything is slower" — a runbook for runtime and database causes

Step	Look at	If you see…	Then…
1	APM Summary: which layer grew?	Database layer	Go to step 5
		Application (Java) layer, all transactions	Go to step 2
		One transaction only	Its trace and breakdown (Day 1)
2	**JVMs** page: heap, GC CPU time	Heap near max, GC time rising, floor rising	Memory leak or undersized heap → heap dump, restart, size the heap
3	**JVMs** page: threads and pools	Pool active = max, threads stuck	Saturation → thread profiler, downstream latency, pool size
4	Infrastructure: host, process, container	CPU high from another process, steal, container at memory limit	Noisy neighbour, VM sizing, container limits
5	**Databases** page: most time consumed	One statement dominates	Slow query trace + explain plan → index / rewrite
		Many calls of a cheap statement	N+1 → batch or join
6	Connection pool and database instance	Pool at max, DB idle / DB connections at limit, locks	Pool sizing, leaks, lock contention
7	Verify	—	Same query `COMPARE WITH` before the fix, at similar traffic

NRQL cheat sheet — runtime (Module 7)

```
-- JVM metric names this agent reports
SELECT uniques(metricTimesliceName) FROM Metric WHERE appName = '<app>' AND
metricTimesliceName LIKE 'Memory/%' SINCE 30 minutes ago
-- Heap used (MB)
SELECT average(newrelic.timeslice.value) FROM Metric WHERE appName = '<app>'
AND metricTimesliceName = 'Memory/Heap/Used' TIMESERIES
-- GC time per collector
SELECT sum(newrelic.timeslice.value) FROM Metric WHERE appName = '<app>' AND
metricTimesliceName LIKE 'GC/%' FACET metricTimesliceName TIMESERIES
-- Host, process and container health
SELECT average(cpuPercent), average(memoryUsedPercent) FROM SystemSample
FACET hostname TIMESERIES
SELECT average(cpuPercent), average(memoryResidentSizeBytes)/1e6 FROM
ProcessSample FACET processDisplayName LIMIT 10
SELECT average(cpuPercent), average(memoryUsageBytes)/1e6, max(restartCount)
FROM ContainerSample FACET name TIMESERIES
-- Capacity trend
SELECT percentile(duration, 95) FROM Transaction WHERE appName = '<app>'
SINCE 4 weeks ago TIMESERIES 1 day
```

NRQL cheat sheet — database (Module 8)

```
-- Database share of each transaction
SELECT average(databaseDuration)/average(duration)*100 AS 'db %',
average(databaseCallCount) FROM Transaction WHERE appName = '<app>' FACET
name
-- Slowest datastore operations
SELECT average(duration.ms), count(*) FROM Span WHERE category = 'datastore'
FACET appName, name LIMIT 20
-- Before vs after a fix
SELECT average(databaseDuration) FROM Transaction WHERE appName = '<app>' AND
name LIKE '%owners%' SINCE 30 minutes ago COMPARE WITH 2 hours ago TIMESERIES
-- PostgreSQL instance
SELECT latest(db.connections), latest(db.commitsPerSecond) FROM
PostgresqlDatabaseSample FACET database
SELECT * FROM PostgresSlowQueries SINCE 30 minutes ago LIMIT 20
```

NRQL cheat sheet — dashboards (Module 9)

```
-- Golden signals, multi-service variable
SELECT rate(count(*), 1 minute) AS 'rpm', percentage(count(*), WHERE error IS
true) AS 'error %',
       percentile(duration, 95) AS 'p95 (s)'
FROM Transaction WHERE appName IN ({{service}}) SINCE 1 hour ago
-- Query variable for the service list
SELECT uniques(appName) FROM Transaction SINCE 1 day ago
-- Trends with comparison
SELECT percentile(duration, 95) FROM Transaction WHERE appName IN
({{service}}) FACET appName TIMESERIES COMPARE WITH 1 week ago
-- SLA
SELECT percentage(count(*), WHERE error IS false) FROM Transaction WHERE
appName = 'easyTravel-frontend' SINCE 30 days ago
```

Knowledge check, answers and sources

Questions

1. Throughput is flat at 300 rpm, response time doubled, CPU is 35 %, and traces show long `getConnection` segments. What is saturated, and what two things would you check?
2. Describe the heap chart of a memory leak, and how it differs from an undersized heap.
3. Why does a GC problem slow down *every* transaction, and why do traces not show a slow segment?
4. Host CPU is 95 %. The JVM's GC time is low. What is the next thing to look at?
5. The heap is limited with `-Xmx1g` and the container memory limit is 1 GB. What will happen under load?
6. On the Databases page, should you rank statements by average time or by most time consumed? Why?
7. What does `record_sql: obfuscated` change, and does it affect explain plans?
8. A query `WHERE last_name LIKE 'Dav%'` scans the whole table although `last_name` has an index. Why might that be on PostgreSQL, and what is the fix?
9. Name two signs of an N+1 query problem in New Relic and the usual fix.
10. When does making a connection pool *smaller* improve performance?
11. Write a billboard query for a multi-value `service` variable showing the p95 latency.
12. What is a linked facet, and how does it help on-call engineers?

Answers

1. The **database connection pool**. Check the pool's active/max on the JVMs page (or the pool's metrics), and the database itself (is it slow or idle?). If the database is idle, the pool is too small or leaking; if it is busy, the queries are the problem.
2. Leak: after every collection the **floor** of the sawtooth is higher than before, until it reaches the maximum and GC runs constantly. Undersized heap: the sawtooth stays close to the maximum but the floor is **level**.
3. GC pauses and the CPU it consumes affect all threads in the JVM, so every request waits a little; the time is spread across ordinary code, not one segment, and appears as "Java" time.
4. `ProcessSample` (which process uses the CPU) and `SystemSample` `cpuStealPercent` — a noisy neighbour or a starved VM.
5. The heap can grow to 1 GB but the JVM also needs memory for metaspace, threads and buffers, so the process exceeds the container limit and is killed (OOM kill, container restart). Keep `-Xmx` well below the limit or use `-XX:MaxRAMPercentage`.
6. **Most time consumed** (average × calls): it measures the total time users wait. A fast query called thousands of times can matter more than a slow rare one.
7. Literal values are replaced with `?` in the SQL sent to New Relic, so personal data and secrets do not leave the application. Explain plans still work — the agent runs the plan against the database itself.
8. With a non-`C` collation (such as `en_US.utf8`), an ordinary B-tree index cannot serve `LIKE` prefix searches. Create an index with the pattern operator class: `CREATE INDEX … ON owners (last_name text_pattern_ops)` (or `varchar_pattern_ops` for `varchar`).
9. A high `databaseCallCount` per transaction; *Identical Calls* in transaction traces; many fast calls adding up to a slow total. Fix by fetching in one query (`JOIN FETCH`, `IN (…)`, batch fetching) or caching.
10. When the **database** is the bottleneck: too many concurrent connections cause CPU thrashing, lock contention and context switching; fewer connections let each query finish faster, and total throughput rises.
11. `SELECT percentile(duration, 95) FROM Transaction WHERE appName IN ({{service}})`
12. A faceted widget whose values can be clicked to **filter the whole dashboard** (or open another dashboard filtered to that value). On-call engineers drill from "top errors" to everything about one error class or one service without writing new queries.

Sources

- New Relic docs — JVMs page (Java): app server metrics from JMX
- New Relic docs — Java agent transaction tracer settings and Slow query details
- New Relic docs — Infrastructure agent as a Docker container and PostgreSQL integration
- New Relic docs — Dashboard template variables
- Dynatrace — easyTravel problem patterns
- Spring — spring-petclinic on GitHub
- PostgreSQL docs — Operator classes and LIKE

JVM metric names and some UI labels vary by agent version; the lab guide shows how to discover the names your agent reports.